DAV 6150 Project 1: Regression Analysis on NY State High School Dropout Metrics

Team/Author(s): <Saugat Sijapati, Sreshta Kusuru, Emmanuel Olimi Kasigazi, Bhavani Danthuri, Nicolette Mtisi>

1. Introduction

The objective of this project is to apply the data science modeling lifecycle to predict the number of student dropouts (dropout_cnt) within specific school district and student subgroup combinations. Since the target variable is a count variable (non-negative integers), Poisson and Negative Binomial regression models will be constructed and compared against a Multiple Linear Regression (MLR) baseline.

```
# Import necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
import statsmodels.api as sm
from statsmodels.genmod.generalized_linear_model import GLM
from statsmodels.genmod import families
from sklearn.metrics import mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore') # Suppress warnings for cleaner output

# Define file paths and key variables
FILE_PATH = 'https://github.com/nic-stack/DAV6150-/raw/refs/heads/main/Project1_Data.csv'
TARGET_VAR = 'dropout_cnt'
EXPOSURE_VAR = 'enroll_cnt'

url = "https://github.com/nic-stack/DAV6150-/raw/refs/heads/main/Project1_Data.csv"
df = pd.read_csv(url)

print(df.shape)
df.head()
```

```
(73152, 29)
```

	report_school_year	aggregation_index	aggregation_type	aggregation_name	nrc_code	nrc_desc	county_code	county_nam
0	2018-19	3	District	ALBANY CITY SCHOOL DISTRICT	3	Urban-Suburban High Needs	1	ALBAN
1	2018-19	3	District	ALBANY CITY SCHOOL DISTRICT	3	Urban-Suburban High Needs	1	ALBAN
2	2018-19	3	District	ALBANY CITY SCHOOL DISTRICT	3	Urban-Suburban High Needs	1	ALBAN
3	2018-19	3	District	ALBANY CITY SCHOOL DISTRICT	3	Urban-Suburban High Needs	1	ALBAN
4	2018-19	3	District	ALBANY CITY SCHOOL DISTRICT	3	Urban-Suburban High Needs	1	ALBAN

5 rows × 29 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 73152 entries, 0 to 73151
Data columns (total 29 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   report_school_year        73152 non-null  object
```

```
 1   aggregation_index           73152 non-null  int64
 2   aggregation_type            73152 non-null  object
 3   aggregation_name            73152 non-null  object
 4   nrc_code                    73152 non-null  int64
 5   nrc_desc                    73152 non-null  object
 6   county_code                 73152 non-null  int64
 7   county_name                 73152 non-null  object
 8   nyc_ind                     73152 non-null  int64
 9   membership_desc             73152 non-null  object
 10  subgroup_code               73152 non-null  int64
 11  subgroup_name               73152 non-null  object
 12  enroll_cnt                  73152 non-null  object
 13  grad_cnt                    73152 non-null  object
 14  grad_pct                    73152 non-null  object
 15  local_cnt                   73152 non-null  object
 16  local_pct                   73152 non-null  object
 17  reg_cnt                     73152 non-null  object
 18  reg_pct                     73152 non-null  object
 19  reg_adv_cnt                 73152 non-null  object
 20  reg_adv_pct                 73152 non-null  object
 21  non_diploma_credential_cnt  73152 non-null  object
 22  non_diploma_credential_pct  73152 non-null  object
 23  still_enr_cnt               73152 non-null  object
 24  still_enr_pct               73152 non-null  object
 25  ged_cnt                     73152 non-null  object
 26  ged_pct                     73152 non-null  object
 27  dropout_cnt                 73152 non-null  object
 28  dropout_pct                 73152 non-null  object
dtypes: int64(5), object(24)
memory usage: 16.2+ MB
```

```
df.shape
```

```
(73152, 29)
```

2. Exploratory Data Analysis (EDA) and Data Preparation (20 Points)

2.1 Data Cleaning and Integrity: Handling Suppression

The raw NYSED data uses the string placeholder '-' for suppressed counts. An initial check for technical nulls reports zero, but inspection reveals object-type columns containing the ' – ' placeholder. This placeholder represents data that is not valid or reliable (cohort size <30 or 40), as per NYSED policy for confidentiality.

Justification: We must convert the placeholders to NaN and then drop all rows where the target is NaN to ensure our model trains only on statistically reliable observations.

Why are there dashes instead of numbers in some places in the New York State Report Cards

```
# 1. Convert placeholders ('-') to NaN and cast to float
numeric_cols = [col for col in df.columns if 'cnt' in col or 'pct' in col]

for col in numeric_cols:
    # Replace the non-numeric suppressor ('-') with NaN and convert to float
    df[col] = pd.to_numeric(df[col].replace('-', np.nan), errors='coerce')

# 2. Filter suppressed data: Drop all rows where the target variable is NaN
df_clean = df.dropna(subset=[TARGET_VAR]).copy()

# 3. Drop non-predictive, redundant, or constant metadata columns
# (Codes, report years, aggregation types, and redundant percentage columns are implicitly excluded later)
df_clean.drop(columns=['report_school_year', 'aggregation_index', 'aggregation_type',
                       'nrc_code', 'county_code', 'subgroup_code', 'membership_desc'],
              inplace=True, errors='ignore')

print(f"\nShape after filtering suppressed data: {df_clean.shape}")
print(f"Total rows dropped due to suppression: {df.shape[0] - df_clean.shape[0]}")
```

```
Shape after filtering suppressed data: (39674, 22)
Total rows dropped due to suppression: 33478
```

2.2 Univariate Analysis: Target Distribution and Overdispersion

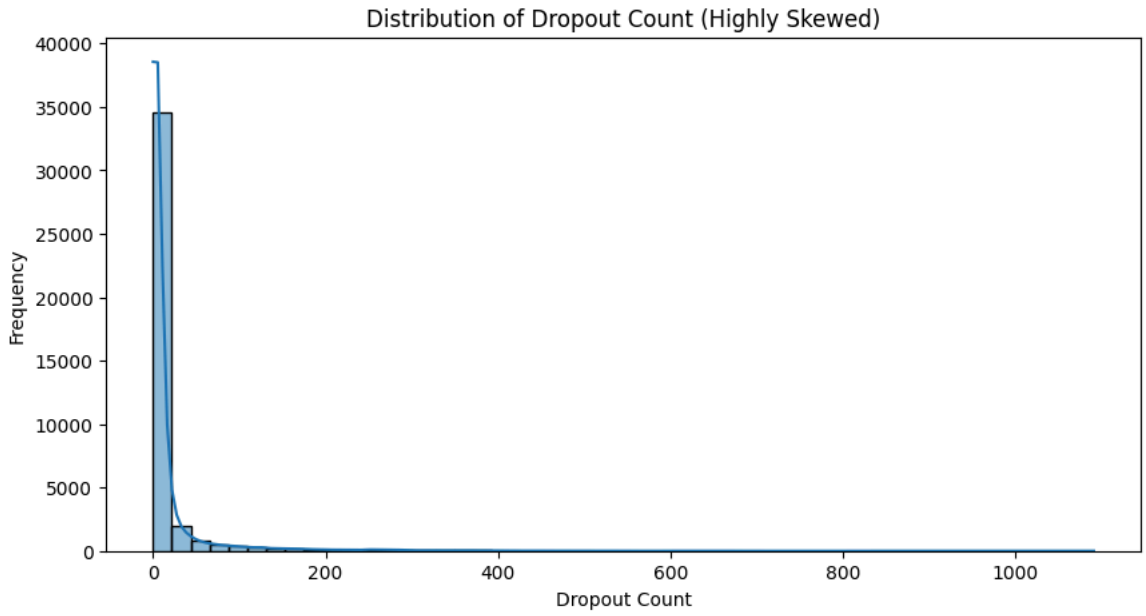
We examine the target variable, dropout_cnt, to check the Poisson assumption (Mean=Variance)

```
mean_y = df_clean[TARGET_VAR].mean()
var_y = df_clean[TARGET_VAR].var()

print("\n--- Target Variable (dropout_cnt) Analysis ---")
print(f"Mean Dropout Count: {mean_y:.2f}")
print(f"Variance Dropout Count: {var_y:.2f}")
print(f"Overdispersion Ratio (Var/Mean): {var_y / mean_y:.2f}")

# Visualization confirms right-skewness
plt.figure(figsize=(10, 5))
sns.histplot(df_clean[TARGET_VAR], bins=50, kde=True)
plt.title('Distribution of Dropout Count (Highly Skewed)')
plt.xlabel('Dropout Count')
plt.ylabel('Frequency')
plt.show()
```

```
--- Target Variable (dropout_cnt) Analysis ---
Mean Dropout Count: 16.24
Variance Dropout Count: 2513.00
Overdispersion Ratio (Var/Mean): 154.75
```

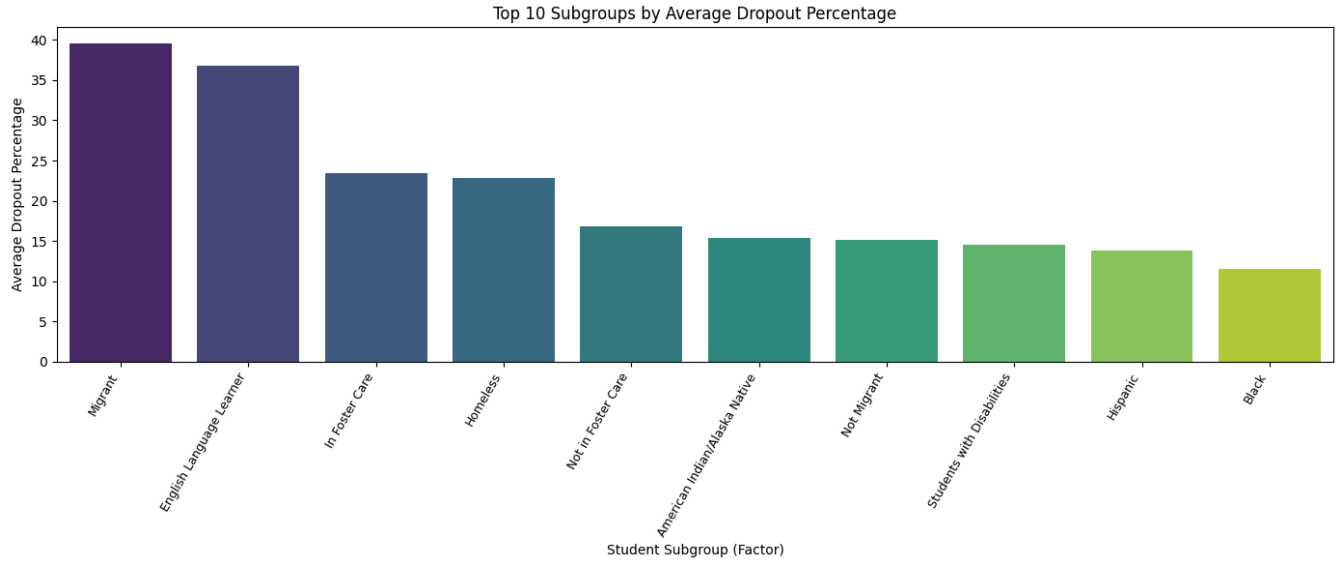


Justification: The Overdispersion Ratio (Variance / Mean) is significantly greater than 1. This indicates the data is highly overdispersed, justifying the necessity of using Negative Binomial regression models.

2.3 Bivariate and Multicollinearity Analysis

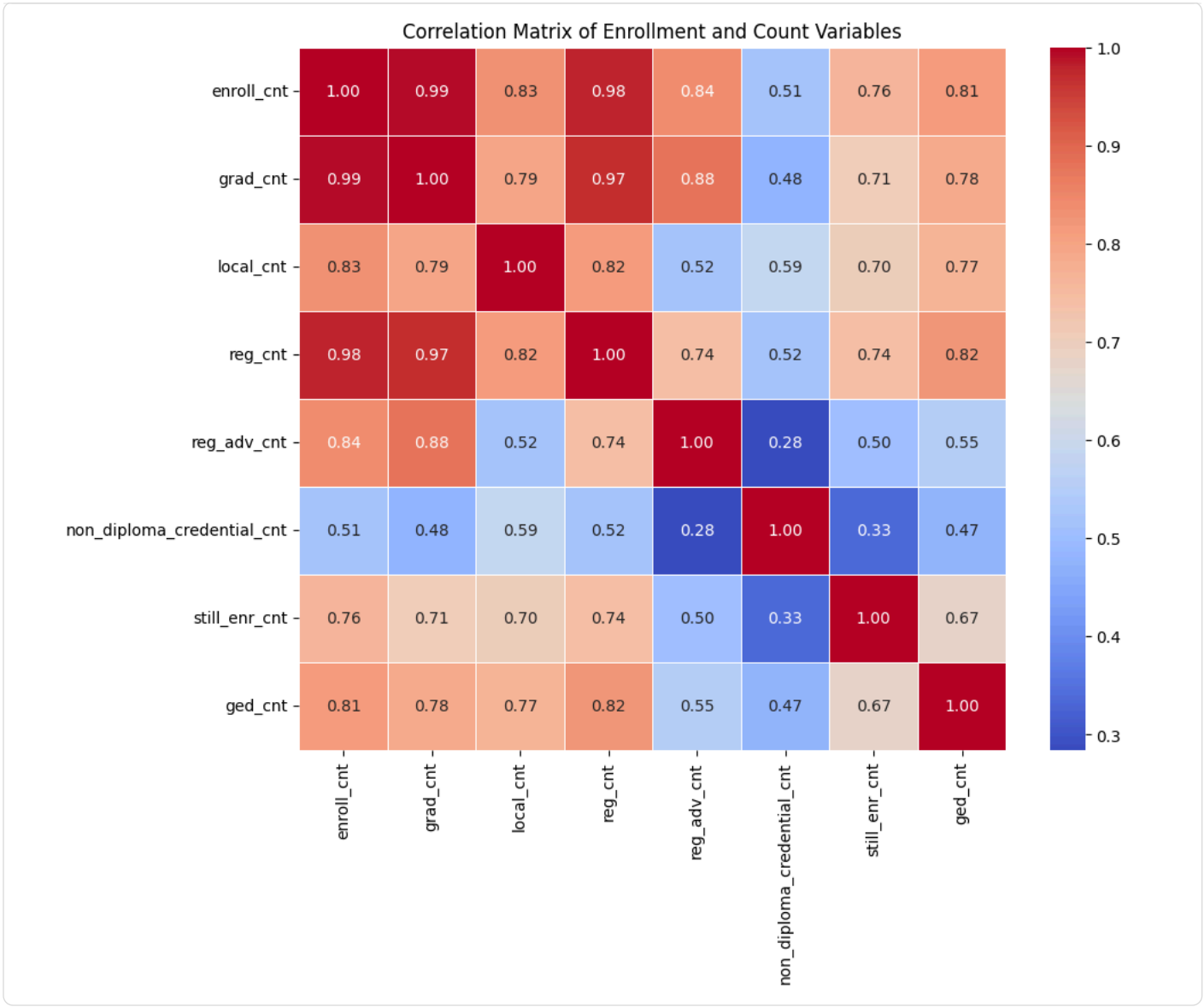
```
# Bivariate Analysis: Dropout Rate by Subgroup
plt.figure(figsize=(14, 6))
subgroup_avg_dropout = df_clean.groupby('subgroup_name')[TARGET_VAR].sum() / df_clean.groupby('subgroup_name')[EXPOSURE
subgroup_avg_dropout = subgroup_avg_dropout[subgroup_avg_dropout.index != 'All Students'].sort_values(ascending=False)

sns.barplot(x=subgroup_avg_dropout.index, y=subgroup_avg_dropout.values, palette='viridis')
plt.xticks(rotation=60, ha='right', fontsize=9)
plt.title('Top 10 Subgroups by Average Dropout Percentage')
plt.ylabel('Average Dropout Percentage')
plt.xlabel('Student Subgroup (Factor)')
plt.tight_layout()
plt.show()
```




```
# Multicollinearity Check: Correlation Matrix
count_cols = [col for col in df_clean.columns if 'cnt' in col and col != TARGET_VAR and col != EXPOSURE_VAR]
corr_matrix = df_clean[[EXPOSURE_VAR] + count_cols].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Matrix of Enrollment and Count Variables')
plt.show()
```



Justification: The high correlation between enroll_cnt and other count variables confirms expected multicollinearity. This is mitigated by using enroll_cnt as the offset, rather than a direct predictor.

3. Feature Engineering and Pre-Modeling Setup (15 Points)

3.1 Feature Engineering: Exposure Variable and Data Splitting

This section details the critical steps taken to prepare the cleaned dataset for Generalized Linear Modeling (GLM). The primary focus is on transforming the target variable to predict the dropout rate and establishing a robust validation framework.

```
# 1. Feature Engineering: Create the Log-Exposure Variable (for GLM Offset)
#Filters rows with zero enrollment to prevent mathematical errors in the logarithm calculation.
df_clean = df_clean[df_clean[EXPOSURE_VAR] > 0].copy()

#Calculates the natural logarithm of the cohort size (enroll_cnt).
df_clean['log_exposure'] = np.log(df_clean[EXPOSURE_VAR])

# 2. Define Final Predictors
# Percentage columns are excluded to prevent redundancy and multicollinearity.
COUNT_PREDICTORS = ['grad_cnt', 'reg_adv_cnt', 'still_enr_cnt']
CAT_PREDICTORS = ['subgroup_name', 'nrc_desc', 'county_name', 'nyc_ind']
ALL_PREDICTORS = COUNT_PREDICTORS + CAT_PREDICTORS

# 3. Define X, y, and the offset
X = df_clean[ALL_PREDICTORS]
y = df_clean[TARGET_VAR].values
offset = df_clean['log_exposure'].values
```

```
# Split data (70% Train, 30% Test)
X_train, X_test, y_train, y_test, offset_train, offset_test = train_test_split(
    X, y, offset, test_size=0.3, random_state=42
)
```

Justification: The core analytical goal is to model the dropout rate, not the raw dropout count. Using log(enroll_cnt) as an offset in the GLM is a necessary statistical technique. It ensures the model adjusts the expected count proportional to the cohort size, allowing us to compare dropout risk rates fairly across districts of all sizes.

log(Expected Count)=β 0+∑β iX i+log(Exposure)

Defining Final Predictors and Excluding Percentages

Predictor Selection: The feature set was curated to include relevant count metrics and contextual categories:

Count Predictors: grad_cnt, reg_adv_cnt, still_enr_cnt (Numerical measures of student success/status).

Categorical Predictors: subgroup_name, nrc_desc, county_name, nyc_ind (Contextual and demographic factors).

Exclusion Rationale: All percentage columns (e.g., grad_pct, dropout_pct) were excluded to prevent high multicollinearity. Since the percentages are derived from the count variables and the enrollment variable (used in the offset), including them would introduce redundancy and negatively impact the stability and interpretation of the model coefficients.

The dataset is partitioned into a 70% Training Set and a 30% Testing Set.

Training Set: Used exclusively to fit and train all regression models.

Testing Set (Holdout): Used only for the final evaluation (Section 5) to provide an unbiased assessment of the preferred model's ability to generalize to unseen data.

The use of a fixed random_state=42 ensures the data split is fully reproducible.

3.2 One-Hot Encoding and Final Data Frame Creation

A ColumnTransformer is used to apply One-Hot Encoding to all categorical features, preparing the data for the regression models.

Justification for One-Hot Encoding:

RRegression models (including MLR, Poisson, and Negative Binomial) are purely mathematical models that operate only on numeric data. They cannot process text or string labels like county names or student subgroup descriptions.

To use the categorical features (like subgroup_name and county_name) in the model, we must use One-Hot Encoding (OHE). OHE is the standard, conservative method for handling Nominal categorical variables (data without inherent order). It converts each unique text label into a separate, binary (0 or 1) numeric column.

Why Not Other Methods?

Label Encoding (e.g., County A=1, County B=2) is rejected because it imposes a false, misleading numerical order on categories that have no inherent rank.

OHE preserves the integrity of the data, ensuring the model treats 'ALBANY County' and 'BRONX County' as distinct factors, not as points on a numerical scale.

```
# --- One-Hot Encoding via ColumnTransformer ---
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), CAT_PREDICTORS),
        ('num', 'passthrough', COUNT_PREDICTORS)
    ],
    remainder='drop'
)

# Fit and Transform Training Data
X_train_processed = preprocessor.fit_transform(X_train)
feature_names = preprocessor.get_feature_names_out()
X_train_df = pd.DataFrame(X_train_processed, columns=feature_names)
X_train_df = sm.add_constant(X_train_df, prepend=False) # Add constant for statsmodels

# Transform Testing Data
X_test_processed = preprocessor.transform(X_test)
X_test_df = pd.DataFrame(X_test_processed, columns=feature_names)
X_test_df = sm.add_constant(X_test_df, prepend=False)
```